3.4 Changing Columns

Like the `remove_column` and `add_column` Rails provides the `change_column` migration method.

```
change_column :products, :part_number, :text
```

This changes the column `part_number` on products table to be a `:text` field.

Besides `change_column`, the `change_column_null` and `change_column_default` methods are used specifically to change the null and default values of a column.

```
change_column_null :products, :name, false
change_column_default :products, :approved, false
```

This sets `:name` field on products to a `NOT NULL` column and the default value of the `:approved` field to false.

Unlike `change_column` (and `change_column_default`), `change_column_null` is reversible.

3.5 Column Modifiers

Column modifiers can be applied when creating or changing a column:

- `limit` Sets the maximum size of the `string/text/binary/integer` fields.
- `precision` Defines the precision for the `decimal` fields, representing the total number of digits in the number.
- `scale` Defines the scale for the `decimal` fields, representing the number of digits after the decimal point.
- `polymorphic` Adds a `type` column for `belongs_to` associations.
- `null` Allows or disallows `NULL` values in the column.
- `default` Allows to set a default value on the column. Note that if you are using a dynamic value (such as a date), the default will only be calculated the first time (i.e. on the date the migration is applied).
- `index` Adds an index for the column.
- `required` Adds `required: true` for `belongs_to` associations and `null: false` to the column in the migration.

Some adapters may support additional options; see the adapter specific API docs for further information.

3.6 Foreign Keys

While it's not required you might want to add foreign key constraints to guarantee referential integrity.

```
add_foreign_key :articles, :authors
```

This adds a new foreign key to the `author_id` column of the `articles` table. The key references the `id` column of the `authors` table. If the column names can not be derived from the table names, you can use the `:column` and `:primary_key` options.

Rails will generate a name for every foreign key starting with `fk_rails_` followed by 10 random characters. There is a `:name` option to specify a different name if needed.

Active Record only supports single column foreign keys. `execute` and `structure.sql` are required to use composite foreign keys. See Schema Dumping and You.

Removing a foreign key is easy as well:

```
# let Active Record figure out the column name
remove_foreign_key :accounts, :branches

# remove foreign key for a specific column
remove_foreign_key :accounts, column: :owner_id
```